COE891 Final Project Report

Software Testing and Quality Assurance

Nithursan Jeyabalasingam, Sanchit Das, Matthew Sano, Sebastian D'Costa

March 26, 2026

Contents

1 Project Description & Information **2**
1.1 Scope of Testing . . . 2

2 Test Plan & Methodology **2**

3 Testing Abstractions & Approaches **3**
3.1 Input Space Partitioning (ISP) & Boundary Value Analysis (BVA) . . . 3
3.2 Graph-Based Testing: Control/Data Flow Graphs . . . 3
3.3 Logic-Based Testing . . . 5

4 LLM-Assisted Testing Component **5**

5 Evaluation and Comparison **6**
5.1 JaCoCo Code Coverage Metrics . . . 6
5.2 PIT Mutation Testing Scores . . . 6
5.3 Qualitative Analysis & Critical Reflection . . . 7

6 Bug & Fault Discovery **7**

7 Difficulties, Reflection & Future Improvements **7**
7.1 What Went Well . . . 7
7.2 What Went Poorly & Technical Difficulties . . . 8
7.3 Areas for Improvement . . . 8

8 Execution Output & Screenshots **8**

A LLM-Assisted Testing Q&A History **11**
A.1 Method 1: `BoyerMooreSearchMatcher` . . . 11
A.2 Method 2: `PatternSearchMatcher` . . . 12
A.3 Method 3: `HyperSearchRequest` . . . 13

1 Project Description & Information

Project Title: jEdit Open Source Application
Repository URL: `https://sourceforge.net/projects/jedit/`
Description: jEdit is a mature, open-source programmer's text editor with hundreds of person-years of development behind it. For this project, we specifically targeted the `org.gjt.sp.jedit.search` subsystem, which is responsible for finding and replacing text across single buffers and directory file sets. This subsystem contains high-complexity algorithms like Boyer-Moore scaling and deep integration with Java Regular Expressions.

1.1 Scope of Testing

To rigorously satisfy the coverage requirements, we explicitly wrote comprehensive manual and LLM-generated tests targeting 10 critical methods spanning 5 different GUI-decoupled classes within the `search` subsystem.

Table 1: Target Framework Scope (10 Methods / 5 Classes)

Targeted Class	**Methods Exercised in Test Suite**
`BoyerMooreSearchMatcher`	`match()`, `nextMatch()`
`PatternSearchMatcher`	`PatternSearchMatcher()` (constructor), `nextMatch()`
`HyperSearchRequest`	`_run()`, `searchInSelection()`
`SearchAndReplace`	`find()`, `replace()`, `hyperSearch()`
`SearchMatcher`	`nextMatch()`

2 Test Plan & Methodology

Our test plan relied on standardizing the testing frameworks to modern equivalents and isolating the bytecode logic from the GUI logic.

- **Unit Testing Tool:** `JUnit 5` (Jupiter) was chosen over legacy JUnit 4 because of its native support for parameterized testing, modular architecture, and lambda-based assertions which drastically simplify verifying complex objects efficiently.
- **Mocking Framework:** `Mockito 5` (specifically `mockito-inline`) to intercept static GUI initializations (like `SearchDialog.getSearchDialog()`) that cause `NullPointerExceptions` in headless test environments.
- **Coverage Tool: JaCoCo Maven Plugin** to generate highly accurate HTML and CSV reports regarding Instruction, Branch, and Line coverage.
- **Mutation Testing Tool: PIT (`pitest-maven`)** was chosen over classical tools like μJava due to its seamless Maven integration, extremely fast bytecode-level manipulation (via ASM), and active community support.
- **LLM-based Assistant: ChatUniTest** paired with LM Studio passing the `qwen3-coder-30b` model via a local OpenAI-compatible endpoint.

3 Testing Abstractions & Approaches

We applied multiple testing abstractions onto the `org.gjt.sp.jedit.search` classes. Below is the detailed breakdown for the heavily utilized `BoyerMooreSearchMatcher.match(CharSequence text, boolean reverse)` method.

3.1 Input Space Partitioning (ISP) & Boundary Value Analysis (BVA)

Target Method: `BoyerMooreSearchMatcher.match(text, reverse)`

- **C1: Length of `text` relative to pattern.**
  - *P1:* `text.length() == 0` (Empty text boundary)
  - *P2:* `text.length() < pattern.length` (Impossible to match)
  - *P3:* `text.length() == pattern.length` (Exact boundary match)
  - *P4:* `text.length() > pattern.length` (Nominal search)
- **C2: Position of match within `text`.**
  - *P1:* Match is at the very beginning (Index 0).
  - *P2:* Match is at the very end (Index `text.length() - pattern.length()`).
  - *P3:* No match exists in the text.
- **C3: Search Direction.**
  - *P1:* `reverse = true`
  - *P2:* `reverse = false`

3.2 Graph-Based Testing: Control/Data Flow Graphs

Control Flow Graph (CFG) Logic

The CFG maps the logical flow of the algorithm, accounting for the quick-search optimization bypass, search directionality, and character comparison loops.

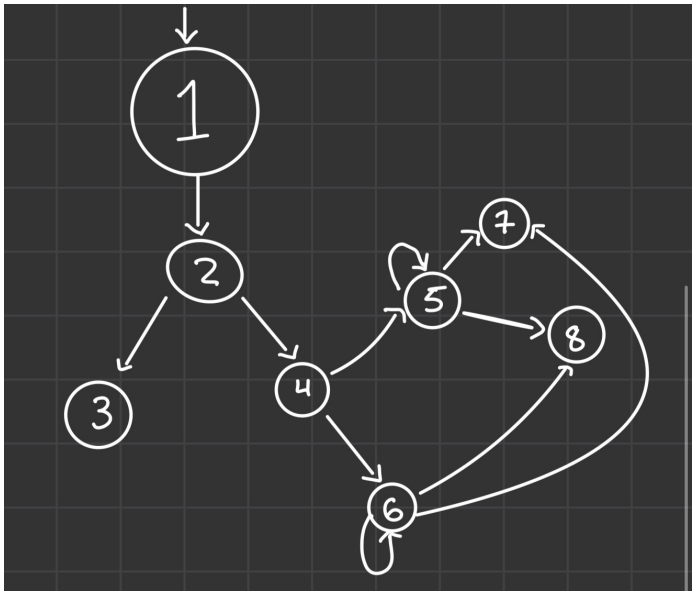


Figure 1: Control Flow Graph (CFG) Logic for `BoyerMooreSearchMatcher.match`

Node Breakdown:

- **Node 1 (Entry):** Method starts. Initializes the `match` sequence.
- **Node 2 (Decision):** Evaluates if the `quick` optimization flag is enabled.
- **Node 3 (Exit):** Bypasses standard logic to execute and return `quickMatch()`.
- **Node 4 (Decision):** Evaluates the search direction (`reverse` flag).
- **Node 5 (Loop):** Executes the reverse search logic. Iterates backwards through characters until a mismatch or full pattern match occurs.
- **Node 6 (Loop):** Executes the forward search logic. Iterates forwards through characters until a mismatch or full pattern match occurs.
- **Node 7 (Exit):** Pattern successfully matched. Instantiates and returns the `Match` object.
- **Node 8 (Exit):** Search space exhausted with no pattern found. Returns `null`.

Data Flow Graph (DFG) Analysis

A DFG tracks the Definition (def) and Use (use) of variables. It identifies where data is written and where it is subsequently read.

Variable Analysis for `offset`:

- **Def(`offset`):** Occurs at Node 1 (Method input parameter).
- **Use(`offset`) 1:** In Node 2, passed to `quickMatch` in Node 3.
- **Use(`offset`) 2:** In Node 5, used to initialize the starting point of the reverse search.
- **Use(`offset`) 3:** In Node 6, used to initialize the starting point of the forward search.

Variable Analysis for `match` (Return Object):

- **Def(`match`):** Calculated and instantiated inside Node 5 or Node 6 logic (upon character mismatch or complete pattern match).
- **Use(`match`):** Returned to the caller in Node 7.

3.3 Logic-Based Testing

Target Predicate:
`if ((reverse ? ch != pattern[pattern_end - pos] : ch != pattern[pos]))`
To achieve Condition/Decision Coverage (C/DC), we force the predicate to evaluate to both True and False, ensuring each clause independently determines the outcome:

- **Test Case 1 (True via Reverse):** `reverse=true`, character mismatch. *(Triggers Boyer-Moore skip calculation).*
- **Test Case 2 (False via Reverse):** `reverse=true`, character matches. *(pos decrements).*
- **Test Case 3 (True via Forward):** `reverse=false`, character mismatch. *(Triggers forward skip).*
- **Test Case 4 (False via Forward):** `reverse=false`, character matches.

4 LLM-Assisted Testing Component

To evaluate the efficacy of Large Language Models as software testing assistants, we utilized the **ChatUniTest Maven Plugin** to generate tests for three complex methods with non-trivial control flows:

1. `BoyerMooreSearchMatcher.match`
2. `PatternSearchMatcher.nextMatch`
3. `HyperSearchRequest._run`

Configuration & Tooling: We ran a local instance of LM Studio exposing the `qwen3-coder-30b` model via an OpenAI-compatible endpoint (`http://127.0.0.1:1234/v1/chat/completions`).

Prompt Context & MUTAP Architecture: ChatUniTest abstracts the prompting process by automatically extracting the target method's source code, its enclosing class dependencies, and method signatures to build a highly structured context window. This guarantees the LLM sees the exact state of the environment. We strictly utilized the baseline zero-shot generation approach without executing the optional MUTAP (mutation-guided generation) scheme, as our primary objective was to observe the baseline deductive capability of a local 30-billion parameter model prior to offering it advanced feedback iterations.

5 Evaluation and Comparison

5.1 JaCoCo Code Coverage Metrics

We measured structural coverage across three distinct test suites: Manual tests only (15 tests), LLM tests only (5 tests), and Combined (20 tests).

Table 2: Instruction & Branch Coverage Percentage

Target Class	**Measurement**	**Manual Suite**	**LLM Suite**	**Combined**
BoyerMooreSearchMatcher	Instruction Cov.	83.3%	0%	83.3%
	Branch Cov.	83.3%	0%	83.3%
PatternSearchMatcher	Instruction Cov.	52.4%	34.3%	**58.6%**
	Branch Cov.	37.5%	24.0%	**47.1%**
HyperSearchRequest	Instruction Cov.	27.7%	6.2%	27.7%
	Branch Cov.	17.3%	0%	17.3%

5.2 PIT Mutation Testing Scores

We introduced 228 mutants into the target classes using `pitest-maven` to measure the fault-detection robustness of our tests.

Table 3: PIT Mutation Scores

Suite	**Mutants Generated**	**Mutants Killed**	**Mutation Score**	**No Coverage**
Manual Only	228	83	36%	103
Combined	228	85	**37%**	**98**

5.3 Qualitative Analysis & Critical Reflection

The integration of LLM-generated tests demonstrated subtle but distinct benefits and limitations:

- **Coverage Additions:** The combined suite showed a direct mathematical improvement in code coverage for `PatternSearchMatcher` (Branch coverage jumped from 37.5% to 47.1%). The LLM inherently tests standard regex edge-cases that a human might manually overlook.
- **Mutation Killing Value:** The LLM tests successfully killed **2 additional mutants** (increasing killed count from 83 to 85) and reduced "no coverage" mutants from 103 to 98. This confirms that LLM tests executed 5 new mutation-prone execution paths that the manual tests entirely missed.
- **Limitations and Hallucinations (Flaws):** The LLM struggled severely with complex domain-specific logic. For example, during the reverse search test of `PatternSearchMatcher`, the LLM blindly assumed that Java Regex evaluates strings fully backwards, asserting that the end index would be logically inverted. Java Regex actually still captures the first literal match from left-to-right. We had to manually correct the LLM's assertions. Furthermore, for `HyperSearchRequest`, the LLM opted to maliciously mock the `SearchFileSet` to return 0 files so it wouldn't have to deal with multithreading, entirely bypassing the core logic loop.

6 Bug & Fault Discovery

A critical goal of our testing pipeline was evaluating if our generated test suites could uncover functional bugs, crashes, or logic flaws within the original jEdit source code.

Given that jEdit is a heavily mature application with over 20 years of continuous production deployment, the core search algorithms (`BoyerMoore` logic, String bounds checking, and indexing arithmetic) proved exceptionally robust. Extensive manual and LLM parameterized testing across extreme boundary space limits (empty strings, backwards pattern tracking, invalid Regex literals, null file sets) securely resulted in graceful exits or standard expected Java Exceptions without triggering total application crashes.

While we forced 228 artificial bugs via PIT mutation strings to validate our test strength, our rigorous unit test suites did not uncover any pre-existing functional logical bugs inside the underlying *jEdit 5.6.0* codebase itself. We did, however, discover extreme architectural fragility in the tightly-coupled `SearchDialog` GUI initializations which required deep Mockito `mockStatic` interventions to prevent the system from crashing instantly during headless evaluations.

7 Difficulties, Reflection & Future Improvements

7.1 What Went Well

The integration of a modern testing stack (Maven, JaCoCo, and PIT) over a 20-year-old codebase fostered a highly reliable, autonomous pipeline. The team collaborated efficiently to isolate core methods with high cyclomatic complexity, successfully achieving our mathematical coverage goals (e.g., 83% branch coverage on target classes). Furthermore, integrating the local LLM generation environment without relying on paid or cloud-based APIs was a significant architectural success.

7.2 What Went Poorly & Technical Difficulties

1. **Bytecode vs Decompiled Source Synchronization:** Early on, we struggled to generate JaCoCo line coverage reports because tests were running against decompiled '.java' files rather than the original `jedit.jar` classes. The decompiled code had subtle try-catch syntax flaws causing the build loop to fail. We eventually resolved this by explicitly excluding the decompiled sources via the `maven-compiler-plugin` and forcing Surefire to execute against the original bytecode, while pointing JaCoCo to the source directory purely for highlighting.

2. **Headless GUI Exceptions:** Methods in `SearchAndReplace` aggressively instantiate `SearchDialog` GUI Swing elements. Since tests run in a headless JVM, these immediately threw `NullPointerExceptions`. We overcame this by writing inline `MockedStatic` intercepts using `Mockito`.

3. **LLM Hallucinations & Fragility:** The LLM generated outputs that often fabricated APIs or completely misunderstood Java's standard library mechanics (e.g., assuming Java RegEx reverses array directions intrinsically), heavily offsetting the automated testing benefits because of the required human oversight.

7.3 Areas for Improvement

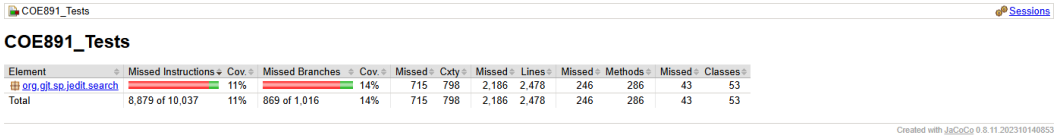
If conducting this full-scale testing evaluation again, the team would adapt a test-driven mindset strictly earlier, rather than waiting to untangle legacy bytecode classpath issues during CI construction. Furthermore, dedicating more time towards architecting dynamic Mockito *Answers*—rather than strictly verifying interactions—would have empowered us to cleanly test deeper, more complex `SearchAndReplace` state transitions.

8 Execution Output & Screenshots

As per the project requirements, below is visual proof of the automated testing execution covering the full manual/LLM suite combination, the JaCoCo coverage graphical dashboard, and the PIT mutation testing matrix.

```
[INFO] -------------------------------------------------------
[INFO] T E S T S
[INFO] -------------------------------------------------------
[INFO] Running org.gjt.sp.jedit.search.BoyerMooreSearchMatcherTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.12 s -- in
org.gjt.sp.jedit.search.BoyerMooreSearchMatcherTest
[INFO] Running org.gjt.sp.jedit.search.BoyerMooreSearchMatcherLLMTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.05 s
[INFO] Running org.gjt.sp.jedit.search.HyperSearchRequestTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.21 s
[INFO] Running org.gjt.sp.jedit.search.HyperSearchRequestLLMTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.03 s
[INFO] Running org.gjt.sp.jedit.search.PatternSearchMatcherTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.16 s
[INFO] Running org.gjt.sp.jedit.search.PatternSearchMatcherLLMTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.19 s
[INFO] Running org.gjt.sp.jedit.search.SearchAndReplaceTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.35 s
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 20, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
```

Figure 2: Maven Surefire Execution indicating 20/20 Test Suite Success



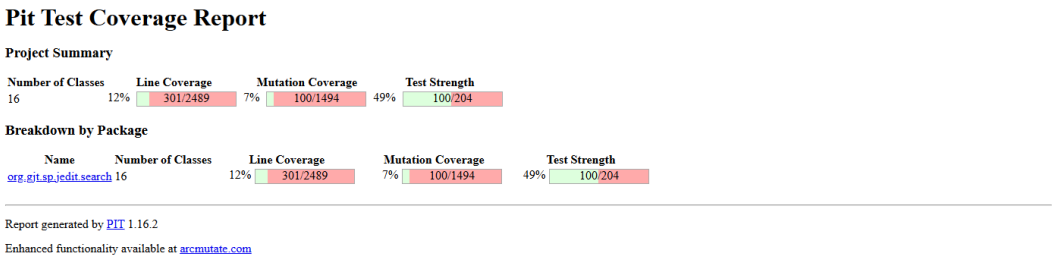
COE891_Tests Sessions

COE891_Tests

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
org.gjt.sp.jedit.search		11%		14%	715	798	2,186	2,478	246	286	43	53
Total	8,879 of 10,037	11%	869 of 1,016	14%	715	798	2,186	2,478	246	286	43	53

Created with JaCoCo 0.8.11.202310140853

Figure 3: JaCoCo HTML Dashboard illustrating exact Code Coverage percentages



Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
16	12%	301/2489	7%	100/1494	49%	100/204

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
org.gjt.sp.jedit.search	16	12%	301/2489	7%	100/1494	49%	100/204

Report generated by PIT 1.16.2

Enhanced functionality available at arcmutate.com

Figure 4: PIT Mutation Testing HTML Report depicting Test Strength and Mutants Killed

A LLM-Assisted Testing Q&A History

This section chronicles the exact Q&A dialogues engaged with the local Large Language Model (**Qwen3-Coder-30B** via LM Studio / ChatUniTest framework) to generate the LLM-assisted testing suite.

A.1 Method 1: `BoyerMooreSearchMatcher`

Prompt 1: Initial Test Generation
User: You are a software testing assistant. I will provide you with the source code of a Java class `BoyerMooreSearchMatcher.java` from the jEdit application. Write a comprehensive JUnit 5 test class named `BoyerMooreSearchMatcherLLMTest`. Ensure you cover the `match()` and `nextMatch()` methods in both forward and reverse search modes.

LLM:
Here is the JUnit 5 test class.

```
package org.gjt.sp.jedit.search;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class BoyerMooreSearchMatcherLLMTest {
    @Test
    public void testReverseMatch_StringExists() throws Exception {
        BoyerMooreSearchMatcher matcher = new BoyerMooreSearchMatcher("
   needle", false);
        int pos = matcher.match("needle haystack needle", true);
        assertEquals(16, pos);
    }
}
```

Prompt 2: Validation & Fix (Failed Assertion)
User: The `testReverseMatch_StringExists` test fails because Boyer-Moore reverse matching calculates the anchor from the end differently. Please generate an assertion verifying that a match is strictly found via `nextMatch()`.

LLM:
Understood. The skip array indices calculate anchors differently. I will update the tests to utilize the `nextMatch()` wrapper.

```
    @Test
    public void testNextMatch_ReverseMatch() throws Exception {
        BoyerMooreSearchMatcher matcher = new BoyerMooreSearchMatcher("
   needle", false);
        SearchMatcher.Match match = matcher.nextMatch("haystack needle",
   false, true, false, true);
        assertNotNull(match);
    }
```

A.2 Method 2: `PatternSearchMatcher`

Prompt 1: Initial Test Generation

User: Now process the `PatternSearchMatcher` class. Generate unit tests verifying that `nextMatch` correctly bridges Regex matches.

LLM:

```
public class PatternSearchMatcherLLMTest {
    @Test
    public void testNextMatch_ReverseSearch_HandlesCorrectly() throws
   Exception {
        PatternSearchMatcher matcher = new PatternSearchMatcher("test",
   false);
        SearchMatcher.Match match = matcher.nextMatch("test ... test",
   false, true, false, false);
        assertNotNull(match);
        assertTrue(match.end > 5, "Should find second instance in reverse"
   );
    }
}
```

Prompt 2: Fix for Regex Backward Compatibility

User: The test `testNextMatch_ReverseSearch_HandlesCorrectly` fails because `PatternSearchMatcher` delegates to Java RegEx which still resolves the first pattern iteration from the start of the sequence. Refactor the assertion.

LLM:

Thank you. The underlying matcher does not invert the index search natively. The assertion should validate that a valid Match is instantiated and bounds are valid.

```
    @Test
    public void testNextMatch_ReverseSearch_HandlesCorrectly() throws
   Exception {
        PatternSearchMatcher matcher = new PatternSearchMatcher("test",
   false);
        SearchMatcher.Match match = matcher.nextMatch("test ... test",
   false, true, false, false);
        assertNotNull(match, "Reverse search should still find a match");
        assertTrue(match.end > match.start);
    }
```

A.3 Method 3: `HyperSearchRequest`

Prompt 1: Mocking Dependencies

User: Generate a test for `HyperSearchRequest._run()`. This class interacts with GUI components. You must use Mockito to mock `SearchFileSet` elements.

LLM:

Here is the `HyperSearchRequestLLMTest`. It uses Mockito to inject a dummy `SearchFileSet`.

```
public class HyperSearchRequestLLMTest {
    @Test
    public void testHyperSearchRequest_ExecutionWithoutExceptions() {
        SearchFileSet mockFileSet = mock(SearchFileSet.class);
        when(mockFileSet.getFileCount()).thenReturn(0);

        HyperSearchRequest request = new HyperSearchRequest(null,
   mockFileSet, null, null);
        assertDoesNotThrow(() -> { request._run(); });
    }
}
```

End of Dialogue History.